

UXL SKILLS EVALUATOR

oneCCL Skill

Collective semantics, launch symmetry, waits, plugins, and hang triage

September 2026 • 5 of 7 evaluation tasks implemented

Why this is useful to oneCCL at first glance

- **Prevent deadlocks before tuning** — Every rank must enter the same collective sequence with compatible buffers, counts, types, roots, and communicators.
- **Make async completion explicit** — A submitted collective is not complete until waited or otherwise synchronized.
- **Collect evidence per rank** — Launch command, mapping, plugin, transport, environment, entry/submission/wait logs, and minimal reproducer.
- **Separate correctness from transport tuning** — Worker count, affinity, plugin, and transport change only after launch symmetry is proven.

What the skill tells an agent today

- **Identify the API surface** — C++ API, NCCL-like C API, PyTorch DDP, Horovod, or framework wrapper.
- **Match collective to semantics** — Allreduce, allgather, alltoall, reduce-scatter, broadcast, point-to-point, or grouped calls.
- **Validate rank symmetry** — Collective order, buffer sizes, datatypes, roots, reductions, and communicator membership.
- **Handle zero counts safely** — Represent empty peer segments without one-past device pointers, including in-place temporaries.
- **Reduce hangs deliberately** — Two ranks, one small tensor, one collective, and logs around entry/submission/wait.

Evaluation inventory: 5 implemented, 2 distributed gaps

- **Implemented** — Async allreduce wait; divergent collective sequence; datatype/count mismatch; C++ versus NCCL-like API; zero-count topo alltoallv.
- **Incident evidence** — The zero-count task checks the accepted repair boundary from upstream issue #174.
- **Planned** — Plugin rank visibility and worker-affinity regression on a stable distributed target.
- **Evidence boundary** — The current fixture does not reproduce the original Aurora/Level Zero failure and must not be presented as hardware qualification.

How we chose the content

- **Official sources** — Project docs, examples, launch guidance, plugins, integrations, releases, and issue evidence.
- **Common distributed failure structure** — Asymmetric calls, waits, buffer/count mismatches, rank mapping, plugin visibility, and hidden framework collectives.
- **Minimal executable tasks** — Hosted fixtures check reasoning and repair contracts where real multi-rank hardware is not required.
- **Visible infrastructure gaps** — Distributed plugin and affinity work stays planned until a faithful lane and provenance exist.

What project ownership would look like

- **Review launch and API guidance** — Especially current C API status, GPU/plugin behavior, framework integration, and environment conventions.
- **Validate hang-triage order** — Ensure the evidence sequence matches how maintainers actually debug incidents.
- **Supply authentic distributed cases** — The highest-value next step is a reproducible plugin or rank-visibility incident.
- **UXL keeps the machinery** — Common evaluator policy, hosted fixtures, dashboards, artifact retention, and runner contracts.

A focused 30-minute maintainer review

- **1. Confirm the first-pass checklist** — API, launcher, ranks, nodes/devices, plugin, transport, buffers, types, counts, and in-place behavior.
- **2. Correct one overstatement** — Especially GPU, plugin, framework, or NCCL-like API assumptions.
- **3. Judge the seven tasks** — Keep, rewrite, or remove; nominate a reproducible distributed incident.
- **4. Name an owner** — A reviewer group for current launch/plugin guidance and incident authenticity.
- **Outcome** — Agents bring a symmetric minimal reproducer and rank-local evidence before asking maintainers to diagnose a hang.